

# Practice Set: Functions, Pointers, and Recursion

Prepared for Practice

## Introduction

This report contains a comprehensive set of 30 programming questions covering:

- Section A: Function-Based Problems (10)
- Section B: Pointer-Based Problems (10)
- Section C: Recursion-Based Problems (10)

Each question includes:

- Problem Description
- Input Format
- Output Format
- C Solution Code

## SECTION A: FUNCTION-BASED QUESTIONS (Easy to Medium)

### Q1. Count Distinct Digits of a Number

Given an integer, count how many **unique** digits appear in it.

**Input:** 552234

**Output:** 3

```
int countDistinct(int n){
    int freq[10] = {0}, count = 0;
    while(n){
        int d = n % 10;
        if(!freq[d]) count++;
        freq[d] = 1;
        n /= 10;
    }
    return count;
}
```

### Q2. First Non-Repeating Character in a String

Return the first character that appears only once.

**Input:** google

**Output:** l

```
char firstNonRepeat(char *s){
    int f[256] = {0};
    for(int i=0;s[i];i++) f[(unsigned char)s[i]]++;
    for(int i=0;s[i];i++)
        if(f[(unsigned char)s[i]] == 1) return s[i];
    return '#';
}
```

### Q3. Convert Seconds to HH:MM:SS Format

Convert time (in seconds) into HH:MM:SS.

**Input:** 3672

**Output:** 01:01:12

```
void convert(int t){
    int h = t/3600;
    int m = (t%3600)/60;
    int s = t%60;
    printf("%02d:%02d:%02d", h, m, s);
}
```

#### Q4. Check if an Array is Strictly Increasing

Verify if each element is strictly larger than the previous.

**Input:** 1 3 5 7 9

**Output:** YES

```
int isIncreasing(int a[], int n){
    for(int i=1;i<n;i++)
        if(a[i] <= a[i-1]) return 0;
    return 1;
}
```

#### Q5. Count Pairs with Sum K

Count all distinct index pairs that sum to a given value.

**Input:** Array: 1 5 7 -1 5 k = 6

**Output:** 3

```
int countPairs(int a[], int n, int k){
    int c = 0;
    for(int i=0;i<n;i++)
        for(int j=i+1;j<n;j++)
            if(a[i] + a[j] == k) c++;
    return c;
}
```

#### Q6. Rotate Array Right by K

Right-rotate an array by K positions.

**Input:** 1 2 3 4 5, k = 2

**Output:** 4 5 1 2 3

```
void rotate(int a[], int n, int k){
    k %= n;
    int t[1000], idx=0;

    for(int i=n-k;i<n;i++) t[idx++] = a[i];
    for(int i=0;i<n-k;i++) t[idx++] = a[i];

    for(int i=0;i<n;i++) a[i] = t[i];
}
```

#### Q7. Check if Two Strings Are Anagrams

Determine whether two strings contain the same characters in any order.

**Input:** listen silent

**Output:** YES

```

int ana(char *a, char *b){
    int f[256]={0};
    for(int i=0;a[i];i++) f[(unsigned char)a[i]]++;
    for(int i=0;b[i];i++) f[(unsigned char)b[i]]--;
    for(int i=0;i<256;i++) if(f[i]!=0) return 0;
    return 1;
}

```

## Q8. Generate All Prefix Sums

Transform array into running prefix sum.

**Input:** 3 1 4 2

**Output:** 3 4 8 10

```

void prefixSum(int *a, int n){
    for(int i=1;i<n;i++)
        a[i] = a[i] + a[i-1];
}

```

## Q9. Print Character Frequencies

Print occurrences of each character in order of first appearance.

**Input:** banana **Output:**

b:1

a:3

n:2

```

void freq(char *s){
    int f[256]={0};
    for(int i=0;s[i];i++) f[(unsigned char)s[i]]++;
    for(int i=0;s[i];i++){
        if(f[(unsigned char)s[i]]){
            printf("%c:%d\n", s[i], f[(unsigned char)s[i]]);
            f[(unsigned char)s[i]]=0;
        }
    }
}

```

## Q10. Find Second Largest Element

Return the second-largest distinct value.

**Input:** 10 20 5 18 20

**Output:** 18

```
int secondLargest(int a[], int n){
    int L1=-1e9, L2=-1e9;
    for(int i=0;i<n;i++){
        if(a[i] > L1){
            L2 = L1;
            L1 = a[i];
        } else if(a[i] > L2 && a[i] != L1){
            L2 = a[i];
        }
    }
    return L2;
}
```

## SECTION B: POINTER-BASED QUESTIONS (Medium to Medium-Hard)

### Q11. Reverse String Using Pointers

Reverse a string in-place using only pointer manipulation.

**Input:** hello

**Output:** olleh

```
void reverse(char *s) {
    char *e = s;
    while(*e) e++;
    e--;

    while(s < e) {
        char t = *s;
        *s = *e;
        *e = t;
        s++; e--;
    }
}
```

### Q12. Print Unique Elements Using Only Pointers

Print elements that appear exactly once.

**Input:** 1 2 2 3 4 4 5

**Output:** 1 3 5

```
void printUnique(int *a, int n) {
    for(int *i=a; i<a+n; i++) {
        int unique = 1;
        for(int *j=a; j<a+n; j++) {
            if(i!=j && *i==*j) {
                unique = 0;
                break;
            }
        }
        if(unique) printf("%d_", *i);
    }
}
```

### Q13. Compare Two Strings via Pointers

Return 1 if equal, else 0.

**Input:** abcd, abce

**Output:** Not Equal

```

int cmp(char *a, char *b) {
    while(*a && *b) {
        if(*a != *b) return 0;
        a++; b++;
    }
    return *a == *b;
}

```

## Q14. Replace Spaces With Hyphens

Modify string in-place using pointer walk.

**Input:** hello world

**Output:** hello-world

```

void replace(char *s) {
    while(*s) {
        if(*s == ' ') *s = '-';
        s++;
    }
}

```

## Q15. Find Minimum and Maximum

Use pointers to return min and max.

**Input:** 4 1 7 2 9

**Output:** 1 9

```

void minmax(int *a, int n, int *mn, int *mx) {
    *mn = *mx = *a;
    for(int i=1; i<n; i++) {
        if(*(a+i) < *mn) *mn = *(a+i);
        if(*(a+i) > *mx) *mx = *(a+i);
    }
}

```

## Q16. Merge Two Sorted Arrays

Use pointers only.

**Input:** a = 1 3 5, b = 2 4 6

**Output:** 1 2 3 4 5 6

```

void merge(int *a, int n, int *b, int m, int *c) {
    int *p=a, *q=b;
    while(p < a+n && q < b+m) {
        *c++ = (*p < *q)? *p++ : *q++;
    }
}

```

```

    }
    while(p < a+n) *c++ = *p++;
    while(q < b+m) *c++ = *q++;
}

```

## Q17. Dot Product of Two Vectors

Compute sum of pairwise products.

**Input:** [1 2 3] [4 5 6]

**Output:** 32

```

int dot(int *a, int *b, int n){
    int s = 0;
    for(int i=0;i<n;i++) s += (*(a+i)) * (*(b+i));
    return s;
}

```

## Q18. Find All Occurrences of a Character

Use pointer indexing.

**Input:** banana, a

**Output:** 1 3 5

```

void occ(char *s, char ch){
    for(int i=0; s[i]; i++){
        if(s[i] == ch) printf("%d_", i);
    }
}

```

## Q19. Pointer-Based Matrix Transpose

Transpose a square matrix using pointer indexing.

**Input:** 1 2 3 4

**Output:** 1 3 2 4

```

void transpose(int (*a)[10], int n){
    for(int i=0;i<n;i++)
        for(int j=i+1;j<n;j++){
            int t = a[i][j];
            a[i][j] = a[j][i];
            a[j][i] = t;
        }
}

```



## Q20. Print Memory Addresses of Array Elements

Display address and value of each integer.

**Input:** 3 7 9

**Output:** 0x7ffd93a3a0 3 0x7ffd93a3a4 7 0x7ffd93a3a8 9

```
void show(int *a, int n){  
    for(int i=0;i<n;i++)  
        printf("%p_%d\n", (a+i), *(a+i));  
}
```

## SECTION C: RECURSION-BASED QUESTIONS (Medium to Hard)

### Q21. Print All Subsets of a String

All possible combinations of characters.

**Input:** abc

**Output:** a, b, c, ab, ac, bc, abc

```
void subsets(char *s, int i, char *curr){
    if(s[i] == '\0'){
        printf("%s\n", curr);
        return;
    }
    subsets(s, i+1, curr);

    int len = strlen(curr);
    curr[len] = s[i];
    curr[len+1] = '\0';
    subsets(s, i+1, curr);
    curr[len]='\0';
}
```

### Q22. Count Number of Zeros Recursively

Count digits equal to zero.

**Input:** 102030040

**Output:** 5

```
int countZero(long n){
    if(n==0) return 0;
    return (n%10==0) + countZero(n/10);
}
```

### Q23. Remove Adjacent Duplicates

Remove repeated characters from a string.

**Input:** aaabbcdadda

**Output:** abcd

```
void removeDup(char *s){
    if(s[0]=='\0' || s[1]=='\0') return;

    if(s[0] == s[1]){
        for(int i=1;s[i];i++) s[i-1]=s[i];
        removeDup(s);
    }
}
```

```
    } else removeDup(s+1);  
}
```

## Q24. Sum of Array (Divide and Conquer)

Return sum of elements.

**Input:** 1 2 3 4 5

**Output:** 15

```
int sum(int *a, int l, int r){  
    if(l == r) return a[l];  
    int m = (l+r)/2;  
    return sum(a,l,m) + sum(a,m+1,r);  
}
```

## Q25. Maximum Depth of Nested Parentheses

Compute nesting depth.

**Input:** (a(b(c)d)(e))

**Output:** 3

```
int depth(char *s, int i, int curr){  
    if(s[i]=='\0') return curr;  
  
    if(s[i]=='(')  
        return depth(s, i+1, curr+1);  
    if(s[i]==')')  
        return depth(s, i+1, curr-1);  
  
    return depth(s, i+1, curr);  
}
```

## Q26. Check If Array Is Sorted Recursively

**Input:** 1 2 4 7 8

**Output:** Sorted

```
int isSorted(int *a, int n){  
    if(n==1) return 1;  
    return (a[n-1] >= a[n-2]) && isSorted(a, n-1);  
}
```

## Q27. Print All Paths in a Maze (Backtracking)

Find all paths from top-left to bottom-right in a binary matrix.

**Input:** 1 1 0 1

**Output:** Path sequence

```
void solve(int a[10][10], int x, int y, int n){
    if(x==n-1 && y==n-1){
        printf("Reached_End\n");
        return;
    }
    if(x+1<n && a[x+1][y]) solve(a,x+1,y,n);
    if(y+1<n && a[x][y+1]) solve(a,x,y+1,n);
}
```

## Q28. Count Ways to Reach Nth Stair

You can climb 1 or 2 steps.

**Input:** 4

**Output:** 5

```
int climb(int n){
    if(n<=2) return n;
    return climb(n-1) + climb(n-2);
}
```

## Q29. Permutations of a String

Print all possible orderings.

**Input:** abc

**Output:** abc acb bac bca ...

```
void perm(char *s, int l, int r){
    if(l==r){
        printf("%s\n", s);
        return;
    }
    for(int i=l;i<=r;i++){
        char t=s[l]; s[l]=s[i]; s[i]=t;
        perm(s,l+1,r);
        t=s[l]; s[l]=s[i]; s[i]=t;
    }
}
```

## Q30. N-Queen Problem (N=4)

Print all valid configurations.

```
int safe(int board[10][10], int r, int c, int n){
    for(int i=0;i<r;i++)
```

```

        if(board[i][c]) return 0;
    for(int i=r, j=c; i>=0&&j>=0; i--, j--)
        if(board[i][j]) return 0;
    for(int i=r, j=c; i>=0&&j<n; i--, j++)
        if(board[i][j]) return 0;
    return 1;
}

void solve(int board[10][10], int r, int n){
    if(r==n){
        for(int i=0; i<n; i++){
            for(int j=0; j<n; j++){
                printf(board[i][j]?"Q_":"._");
            }
            printf("\n");
        }
        return;
    }
    for(int c=0; c<n; c++){
        if(safe(board, r, c, n)){
            board[r][c]=1;
            solve(board, r+1, n);
            board[r][c]=0;
        }
    }
}

```